TP 03 — Résoudre un labyrinthe

Inside ESILV : une journée dans la peau d'un élève-ingénieur

Sixtine Michel

De Vinci Higher Education — ESILV

ESILV
DE VINCI ENGINEERING SCHOOL

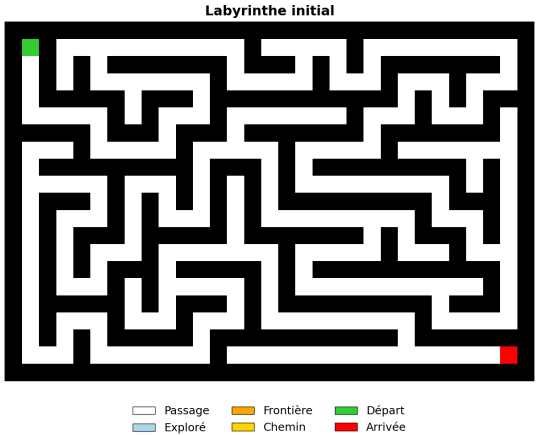
Notre mission

Objectif

Générer automatiquement un labyrinthe, puis trouver un chemin entre le départ vert et l'arrivée rouge.

Nous utiliserons deux structures :

- une **pile** pour générer le labyrinthe ;
- une **file** pour chercher le chemin.



D'un point de vue algorithmique

Le programme sépare la création du problème et sa résolution :

```
def creer_labyrinthe(nb_lignes, nb_colonnes):
    # Creuser les passages avec une pile.
def preparer_resolution(grille, depart, arrivee):
    # Initialiser la file du parcours en largeur.
def etape_bfs(grille, etat):
    # Explorer une case et ajouter ses voisines.
def construire_image(grille, etat):
    # Colorer exploration, frontiere et chemin.
```

L'animation appelle `etape_bfs` jusqu'à atteindre l'arrivée.

Représenter le labyrinthe

Le labyrinthe est un tableau NumPy à deux dimensions :

$$\texttt{grille[ligne, colonne]} = \begin{cases} 0 & \text{si la case est un mur,} \\ 1 & \text{si la case est un passage.} \end{cases}$$

Coordonnées

Une case est représentée par un tuple `(ligne, colonne)`.

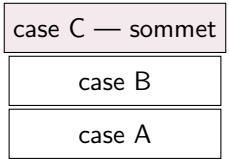
Déplacements

Une direction est un tuple `(dl, dc)` ajouté aux coordonnées.

Pile et file : deux ordres différents

Pile — génération

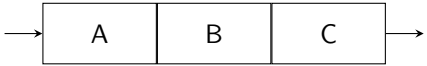
Dernier entré, premier sorti.



`pile.append(C)` puis `pile.pop()`

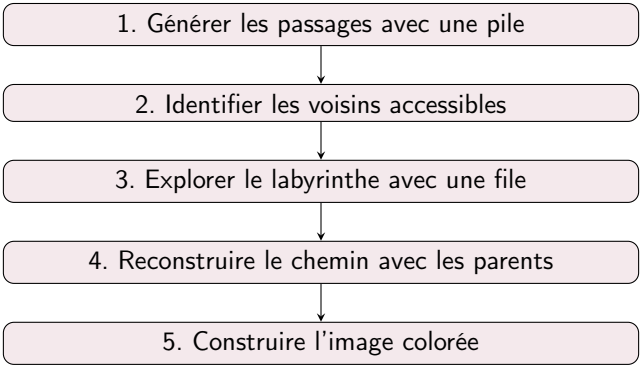
File — résolution BFS

Premier entré, premier sorti.



`file.append(C)` puis `file.popleft()`

Les cinq étapes du programme



Correspondance entre les étapes et les TODO

1. **Générer les passages avec une pile**	**TODO 1 à 5**
2. **Identifier les voisins accessibles**	**TODO 6 à 8**
3. **Explorer avec une file BFS**	**TODO 9, 11 et 12**
4. **Reconstruire le chemin avec les parents**	**TODO 10**
5. **Construire l'image colorée**	**TODO 13 à 15**

Partie fournie

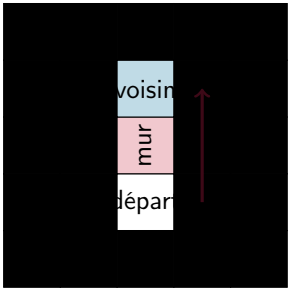
La création de la fenêtre et l'animation Matplotlib sont déjà écrites.

Générer : avancer de deux cases

Une cellule logique sur deux est séparée par un mur. Pour aller de (l, c) à $(l + 2, c)$, il faut ouvrir :

- le mur intermédiaire $(l + 1, c)$;
- la nouvelle cellule $(l + 2, c)$.

Les dimensions impaires conservent une bordure de murs.



Directions et candidats — TODO 1 à 3

À vous

- Definir les quatre directions de generation.
- Calculer les coordonnees (nl, nc) du candidat.
- Conserver uniquement un candidat valide.

Directions et candidats — TODO 1 à 3

```python
# TODO 1 : sauter de deux cases
directions = [(-2, 0), (2, 0), (0, -2), (0, 2)]

for dl, dc in directions:
    # TODO 2 : coordonnees du candidat
    nl = ligne + dl
    nc = colonne + dc

    # TODO 3 : interieur et non visite
    if (
        1 <= nl < nb_lignes - 1
        and 1 <= nc < nb_colonnes - 1
        and grille[nl, nc] == MUR
    ):
        voisins.append((nl, nc, dl, dc))
```

Creuser et avancer — TODO 4 et 5

À vous

- Choisir aleatoirement un candidat dans voisins.
- Creuser puis avancer.

Creuser et avancer — TODO 4 et 5

```
# TODO 4 : choix aleatoire d'un voisin valide
nl, nc, dl, dc = random.choice(voisins)

# TODO 5 : ouvrir le mur situe a mi-chemin
grille[ligne + dl // 2, colonne + dc // 2] = PASSAGE

# Ouvrir le voisin puis le placer au sommet de la pile
grille[nl, nc] = PASSAGE
pile.append((nl, nc))
```

Retour en arrière

Sans voisin disponible, `pile.pop()` retire la case courante et reprend la génération depuis la case précédente.

Trouver les passages voisins — TODO 6 à 8

À vous

- Definir les quatre deplacements du parcours.
- Calculer les coordonnees de la case voisine.
- Verifier que la case est dans la grille et n'est pas un mur.

Trouver les passages voisins — TODO 6 à 8

```python
# TODO 6 : deplacements d'une seule case
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

for dl, dc in directions:
    # TODO 7
    nl = ligne + dl
    nc = colonne + dc

    # TODO 8
    if (
        0 <= nl < grille.shape[0]
        and 0 <= nc < grille.shape[1]
        and grille[nl, nc] != MUR
    ):
        voisins.append((nl, nc))
```

Une file pour le BFS — TODO 9

Le BFS traite toujours la case présente depuis le plus longtemps :

Une file pour le BFS — TODO 9

Le BFS traite toujours la case présente depuis le plus longtemps :

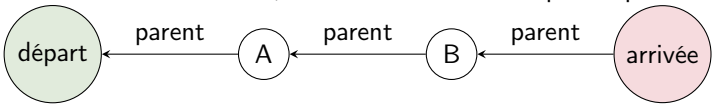
```
# Retirer a gauche : premier entre, premier sorti
case = etat["file"].popleft()
```

Pourquoi obtient-on un chemin minimal ?

La file explore d'abord toutes les cases à distance 1, puis toutes celles à distance 2, et ainsi de suite. La première arrivée trouvée utilise donc le plus petit nombre de déplacements.

Mémoriser les parents

Lorsqu'une nouvelle case est découverte, on mémorise la case depuis laquelle elle a été atteinte.



À l'arrivée, il suffit de suivre ces flèches en sens inverse.

Reconstruire le chemin — TODO 10

À vous

- Reconstruire le chemin.

Reconstruire le chemin — TODO 10

```python
chemin = []
courant = case

while courant is not None:
    chemin.append(courant)
    courant = etat["parents"][courant]

# La liste va de l'arrivee vers le depart : on l'inverse.
etat["chemin"] = chemin[::-1]
```

Planifier un nouveau voisin — TODO 11 et 12

À vous

- Traiter uniquement un voisin encore jamais visite.
- Memoriser puis planifier ce voisin.

Planifier un nouveau voisin — TODO 11 et 12

```python
for voisin in voisins_accessibles(grille, case):
    # TODO 11 : ne jamais visiter deux fois la meme case
    if voisin not in etat["visites"]:
        # TODO 12 : memoriser et planifier
        etat["visites"].add(voisin)
        etat["parents"][voisin] = case
        etat["file"].append(voisin)
```

Ajouter une case à `visites` dès sa découverte évite de l'insérer plusieurs fois dans la file.

Construire l'image — TODO 13 à 15

L'image est une copie de la grille dans laquelle chaque état reçoit un code.

Construire l'image — TODO 13 à 15

L'image est une copie de la grille dans laquelle chaque état reçoit un code.

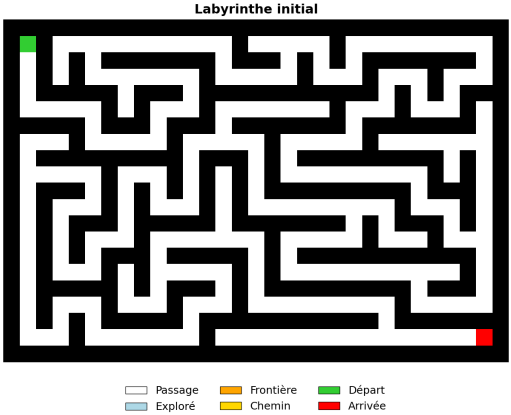
```
# TODO 13 : cases explorees
if image[ligne, colonne] != MUR:
    image[ligne, colonne] = EXPLORE

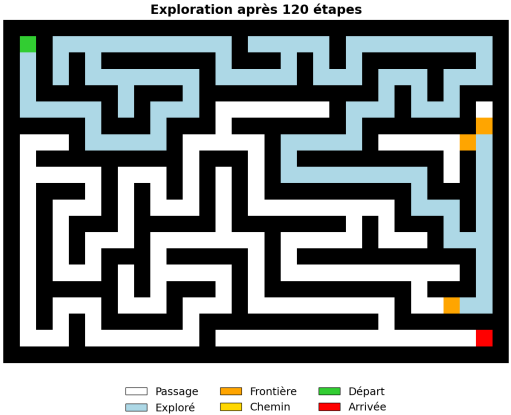
# TODO 14 : cases en attente dans la file
image[ligne, colonne] = FRONTIERE

# TODO 15 : solution finale
image[ligne, colonne] = CHEMIN
```

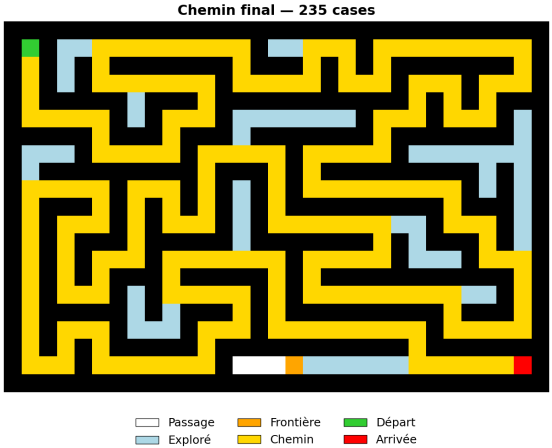
Le départ et l'arrivée sont colorés en dernier pour rester toujours visibles.

Observer l'exploration





Chemin final



Résultat

Le chemin jaune relie le départ à l'arrivée. Dans ce labyrinthe parfait, il est unique.

La zone bleue contient les cases retirées de la file. La zone orange est la frontière restant à explorer.

Expérimenter

Après avoir obtenu le chemin, modifier un seul élément :

- augmenter `NB_LIGNES` et `NB_COLONNES` ;
- remplacer `popleft()` par `pop()` et comparer l'exploration ;
- changer l'ordre des quatre directions ;
- déplacer le départ ou l'arrivée sur une autre case de passage.

Question

Le chemin trouvé change-t-il ? Le nombre de cases explorées change-t-il ?

Lancer le TP dans Spyder

1. Ouvrir `03-resolution_labyrinthe.py`.
2. Choisir un backend graphique **Automatic** ou **Qt**.
3. Redémarrer la console IPython si le backend vient d'être modifié.
4. Compléter les TODO dans l'ordre, puis exécuter après chaque groupe.

Animation dynamique

`plt.show(block=True)` maintient la fenêtre ouverte pendant le BFS.

Concepts appris

- tableau 2D et coordonnées ;
- tuples et listes ;
- pile et retour en arrière ;
- conditions sur une grille ;
- file deque ;
- parcours en largeur BFS ;
- dictionnaire des parents ;
- reconstruction d'un chemin.

À vous de coder !